

GAME NETWORKING - LEC

Lab Activity 2: Creating a Live Multi-Client Global Chat Box

Name: ____

Lab Activity: Creating a Live Multi-Client Global Chat Box (With Partner)

Lab Activity 2: The Multi-Client Global Chat Box (The RPC Network Bridge)

- **Business Scenario:**

The game designers at **ForgePixel Games** aim to implement a real-time global chat window within the multiplayer hub of The Last Spark, enabling players to coordinate team strategies.

- **Technical Task:**

You and your partner will create a functional text window layout in Unity. You will write an **RPC Communication script** (C#) that captures local text string inputs from a text box, forwards the package up to the host server using a `ServerRpc`, validates the text layout formatting on the server, and securely echoes the message down to all connected players simultaneously using a corresponding `ClientRpc`.

- **Focus:** Mastering string communication over network sockets and managing multi-client UI updates without desync bugs.

Workflow:

Step 1: Design the UI Canvas Layout

This step can be built using Unity's Legacy UI

1. Open your active multiplayer scene layout from your project.
2. Right-click inside your Hierarchy window and select **UI > Canvas**. Rename this object to `Chat_Canvas`.
3. Right-click on `Chat_Canvas` and add a **Panel** (**UI > Panel**). Move, scale, and anchor this panel to the lower-left corner of the screen to serve as the chat box container frame.
4. Right-click inside your new Panel and add the following three visual components [Defining the Chat Box in Module 3]:
 - **Scroll View (UI > Scroll View):** Scale this to occupy the top 80% of the panel. Inside its internal hierarchy, delete the horizontal scrollbar. Locate the Content object child inside it, and attach a **Vertical Layout Group** component to it. This causes text entries to stack cleanly.
 - **Input Field (UI > Input Field):** Place this at the bottom-left of the panel. Rename it to `Chat_InputField` [Defining the Chat Box in Module 3].
 - **Button (UI > Button):** Place this directly next to the input box at the bottom-right. Change its child text to read "Send". Rename the button object to `Button_Send`.

Date: August 3, 2026

5. Create a new **Text** object inside the Scroll View's Content container slot. Scale it wide to fit the panel box. Rename this text component to Text_ChatDisplayBox.

Step 2: Write the RPC Communication Script

1. Go to your Project Assets tab window, right-click, and select Create > C# Script. Name this script exactly **GlobalNetworkChatManager** [Global Network Chat Manager].
2. Open the script file inside your code editor and construct the network routing channels using this verified C# RPC architecture layout [Global Network Chat Manager]:

Step 3: Map Component Fields inside the Unity Inspector

1. Return to your main scene view window in the Unity Editor interface.
2. Select your master network engine object (the _NetworkManager GameObject template built during your previous class milestones) [Step 1: NGO System Provisioning].
3. Drag and drop your completed **GlobalNetworkChatManager** script file from your project assets window directly into the Inspector panel array list of _NetworkManager to mount the component [Step 2: Refactoring the Character Asset Prefab].
4. Look at the three exposed component public slot fields under the script header settings inside the Inspector drawer [Global Network Chat Manager]. Link them up by dragging objects from the Hierarchy directly into their targeting slot lines:
 - Drag your Chat_InputField object into the **Chat Input Field** variable box slot.
 - Drag your Text_ChatDisplayBox object into the **Chat Display Area Box** variable box slot.
 - Drag your Button_Send object into the **Send Text Button** variable box slot.
5. Save your scene changes layout completely (Ctrl + S).

Step 4: Live Viewport Multi-Client Execution Check

To ensure the multi-client network chat system behaves accurately without text duplicating or dropping variables along the pipeline, pairs should complete this verification test matrix:

1. Open your workspace compilation paths panel drawer and select File > Build Settings.... Click **Build and Run** to initialize a standalone game build workspace window (.exe) on your desktop canvas layout [Execution Test Sequence Steps].
2. Move the .exe package window over to the left side of your screen. Next, go back into your main Unity Editor development tool screen and click the standard **Play Button** to start a secondary runtime layout thread [Execution Test Sequence Steps]. Move this window to the right [Execution Test Sequence Steps].
3. Click the visual HUD control buttons to set Instance A (The standalone build) to **Host Session**, and set Instance B (The Unity Editor) to **Join Match** [Success Audit Validation Gates].

Date: August 3, 2026

4. **The Network Data Validation Test:** Left-click inside the text entry slot on Instance B (Player 1 Client window), type "En route to alpha waypoint, coordinates verified.", and hit your custom **Send** UI button [Is a Chat Box Possible Without a Database?].

The Success Verification Gate: Look at both running application windows simultaneously. The typed string text line must display instantly inside the chat area boxes of **both screens simultaneously**, confirming that your volatile C# RPC multi-client message network framework is success-verified and active

Deliverables List:

Student groups must compile their project assets into a single zipped archive named Module3_Lab1_DatabaseLessChat_YourNames.zip containing the following three (4) core files:

1. **The Core C# Network Messaging Script:** GlobalNetworkChatManager.cs
 - *Requirement:* The file must contain your complete C# script, showcasing both the [ServerRpc(RequireOwnership = false)] entry validation logic and the matching [ClientRpc] broadcast loops.
2. **Operational Standalone Desktop Build (\Build\ Folder):**
 - *Requirement:* A compiled Windows Executable application package (.exe). The build must run independently of the Unity Editor and must contain your Chat_Canvas interface layouts mapped cleanly to the active Network Manager socket transport layers.
3. **The Layout UI Screenshot:** Lab1_UI_Hierarchy_YourNames.png (or .jpg)
 - *Requirement:* A clear viewport screenshot of the Unity Editor. It must display your Hierarchy window expanded to show the full tree layout structure of the Chat_Canvas components (Scroll View, Content, Input Field, and Send Button) mapped to the inspector fields.
4. **Live Synchronization Demonstration Video:** Lab2_ChatDemo_YourNames.mp4
 - *Requirement:* A short screen recording (maximum 30 seconds) displaying **two separate instances of your game build running side-by-side on your desktop monitor**. You must type a message into one window, click send, and show that the text displays instantly on *both* screens simultaneously with unique Player Network ID tags.

Due date: August 10, 2026